

IPL Analytics Dashboard

A Step-by-Step Technical Guide for Building Premium Power BI Solutions and Showcasing Full-Stack Analytics Competency to Recruiters

Author: Data Portfolio Project

Database Engine: PostgreSQL (ipl_db)

Target Platform: Microsoft Power BI Desktop

Coverage: Historical IPL Data (2008–2026)

Generated June 2026

1. Visual Design & Aesthetics

To impress recruiters, your dashboard must look like a professional corporate product rather than an amateur school project. Follow these design aesthetics:

- **Harmonious Color Palette:** Avoid bright, primary colors. Use a sleek dark mode theme (background Hex: #0F111C or #121422) with single neon accents (e.g. Neon Cyan #00F2FE for metrics, Neon Pink #FF3366 for highlights). This creates contrast and catches the eye immediately.
- **Consistent Typography:** Use a modern, clean font such as *Segoe UI* or *Segoe UI Semibold* throughout the dashboard. Do not mix more than two font families.
- **Glassmorphism and Cards:** Structure your visuals inside clean card panels with rounded corners (border-radius: 8-12px) and a slightly lighter dark background than the canvas. Add a subtle border to create a layered, premium structure.
- **Clear Spacing:** Allow breathing room. Leave consistent 10-15px gaps between charts (padding). Clutter is the number one sign of an amateur developer.

2. Database Connection & Data Modeling

Connecting directly to a relational database like PostgreSQL demonstrates real-world database integration skills, which is highly valued by recruiters.

PostgreSQL Connection Settings:

1. Click **Get Data** in Power BI Desktop and select **PostgreSQL database**.
2. Enter **Server:** localhost and **Database:** ipl_db.
3. Under authentication, choose **Database**, input Username: postgres, Password: root, and connect.

Establishing the Relational Schema:

In the Power BI **Model View**, you must configure a relational model. Drag match_id from the matches table and drop it onto match_id in the deliveries table. Power BI will establish a **One-to-Many (1 to *) relationship**. Set the Cross-Filter Direction to **Single** (meaning matches filters deliveries, protecting query performance).

3. Step-by-Step Page Configuration

Set up a multi-tab report with clear visual focal points on each page:

Page 1: Overview (The executive summary)

- **Main KPI Cards:** Total Matches (Count of matches[match_id]), Total Seasons (Distinct Count of matches[season]), and Total Runs (Sum of deliveries[total_runs]). Place these side-by-side at the very top.
- **Matches per Season (Line Chart):** Year-on-year growth. Axis: season, Values: match_id (Count).
- **Team Wins (Clustered Horizontal Bar Chart):** Y-Axis: winner, X-Axis: match_id (Count). Order descending to show the most successful team immediately.

Page 2: Player Analysis (Leaderboards)

- **Top 10 Batsmen (Bar Chart):** Y-Axis: batsman, X-Axis: batsman_runs (Sum). Use visual level filters to restrict the top 10.
- **Top 10 Bowlers (Bar Chart):** Y-Axis: bowler, X-Axis: id (Count, renamed to Wickets). Add a visual-level filter on dismissal_kind to exclude non-bowler wickets: only keep *bowled*, *caught*, *caught and bowled*, *lbw*, *stumped*, *hit wicket*.

Page 3: Toss Analysis

- **Toss Decision Preference (Donut Chart):** Legend: toss_decision, Values: match_id (Count).
- **Toss Outcome vs. Match Outcome (Pie Chart):** Show if winning the toss correlates with winning the match. Legend: Custom column Toss Match Win, Values: match_id (Count).

4. Advanced DAX Measures

Recruiters evaluate your technical depth based on your DAX coding. Avoid default aggregations. Write custom measures for advanced KPIs:

DAX Measure: Batsman Strike Rate (SR)

A simple runs-per-ball division. To be accurate, we must filter out wide deliveries, as they do not count as legal balls faced by a batsman:

```
Batsman Strike Rate = VAR RunsScored = SUM(deliveries[batsman_runs]) VAR  
LegalBallsFaced = CALCULATE(COUNTROWS(deliveries), deliveries[wides] = 0)  
RETURN DIVIDE(RunsScored, LegalBallsFaced, 0) * 100
```

DAX Measure: Bowler Economy Rate

Economy is runs conceded per over. We count legal deliveries thrown and divide by 6 to represent overs bowled, handling divisions by zero gracefully:

```
Bowler Economy = VAR RunsConceded = SUM(deliveries[total_runs]) VAR  
LegalBallsBowled = CALCULATE(COUNTROWS(deliveries), deliveries[wides] = 0,  
deliveries[noballs] = 0) VAR OversBowled = DIVIDE(LegalBallsBowled, 6, 0)  
RETURN DIVIDE(RunsConceded, OversBowled, 0)
```

5. Recruiter & Portfolio Strategy

To make your project stand out to recruiters, you must frame your dashboard within a wider software engineering pipeline. Use the following structure on your resume and GitHub:

- **Showcase End-to-End Pipeline Competency:** Do not just show a dashboard. Highlight that you built the entire ETL pipeline: Excel ➡■ Pandas Data Cleaning ➡■ PostgreSQL Database Ingestion ➡■ SQL Analytics ➡■ Scikit-Learn Machine Learning ➡■ Power BI & Streamlit UI. This demonstrates full-stack data capability.
- **Document the Database Schema:** Detail your database creation in your project documentation, explaining how you mapped match IDs using a fuzzy date tolerance algorithm (+/- 1 day) in Python to connect deliveries and matches tables. This shows advanced problem-solving skills.
- **Highlight DAX Complexity:** Write in your portfolio about your custom DAX equations (e.g. Strike Rate and Bowler Economy). Explain how you handled cricket-specific logic (e.g., filtering out wides for balls faced, and calculating overs correctly). This shows attention to business logic.
- **Embed the Interactive Streamlit App:** Link to the Streamlit web predictor from your Power BI dashboard or GitHub. Recruiter feedback is highly positive when they see that a user can interact with an ML model directly through a polished, web-based UI.

Recruiter Interview Checklist:

Topic	Key Portfolio Selling Point
Data Extraction & ETL	Wrote ETL scripts in Python to merge 1,243 ball-by-ball files and load 295K rows into PostgreSQL in under 15 seconds.
Database Design	Designed relational schema in PostgreSQL with primary/foreign keys to represent matches and deliveries.
Machine Learning	Trained a Random Forest Classifier to predict match winners, serving probability outputs directly in Streamlit.
Advanced DAX	Wrote clean, custom DAX metrics for batsman strike rate and bowler economy, handling edge cases like extras and wides.